AGENDA PRO

Sistema de Agendamento de Serviços

Especificação funcional, técnica e plano de execução

Projeto de portfólio com foco em backend .NET, testes automatizados e frontend React desenvolvido pelo próprio autor.

ESCOPO CONTROLADO • 5 TABELAS • API REST • MYSQL • REACT

Versão 2.0 — planejamento consolidado

Documento de trabalho: requisitos e decisões podem ser refinados antes da implementação.

Como usar este documento

Este documento consolida o plano inicial do projeto e as decisões adicionais: API construída primeiro, testes unitários desde o início, MySQL criado por migrations, Swagger para exploração manual da API e frontend em React com JavaScript desenvolvido de forma independente pelo autor, com orientação pontual.

Princípio central: implementar pouco por vez, testar cada comportamento e manter o escopo deliberadamente pequeno.

Sumário

1. Introdução, contexto e objetivo
2. Problema, público e limites do sistema
3. Requisitos funcionais (RF) e não funcionais (RNF)
4. Regras de negócio (RN)
5. Arquitetura e organização da solution
6. Modelo de dados: cinco tabelas
7. Contrato da API e códigos HTTP
8. Estratégia de testes e casos de teste
9. Plano de frontend React
10. Tecnologias e ferramentas
11. Fases de desenvolvimento e critérios de aceite
12. Git, documentação e entrega
13. Riscos, decisões em aberto e fora de escopo

1. Introdução, contexto e objetivo

1.1 Introdução

AgendaPro é um sistema de pequeno porte para organizar serviços com hora marcada. A aplicação permite manter cadastros de clientes, profissionais e serviços, criar e administrar agendamentos, registrar pagamentos e consultar informações operacionais. O projeto será desenvolvido como uma aplicação monolítica modular: uma solução .NET dividida em projetos por responsabilidade, implantada como uma aplicação, sem microserviços.

1.2 Objetivo geral

Construir um sistema funcional e demonstrável que evidencie domínio de desenvolvimento backend em C#/.NET, modelagem relacional, regras de negócio, API REST, testes automatizados, persistência com migrations e tratamento consistente de erros. Como objetivo complementar, construir um frontend React com JavaScript, funcional e visualmente organizado, implementado pelo próprio autor para consolidar fundamentos de interface.

1.3 Objetivos específicos

- Modelar e persistir exatamente cinco tabelas principais, com chaves estrangeiras e restrições coerentes.
- Separar domínio, casos de uso, infraestrutura e camada HTTP, evitando dependências invertidas.
- Implementar validações e regras de negócio nos Services/casos de uso, sem depender do frontend para garantir integridade.
- Cobrir regras críticas com testes unitários automatizados e fluxos prioritários com testes de integração.
- Disponibilizar Swagger/OpenAPI para inspecionar e executar requisições manualmente.
- Usar migrations para criar e evoluir o schema do MySQL de forma reproduzível.
- Implementar telas React simples, responsivas e acessíveis o suficiente para operar os fluxos principais.
- Documentar decisões, configuração, execução e limitações no README.

1.4 Critério de sucesso

O projeto será considerado bem-sucedido quando os fluxos principais funcionarem, as regras críticas estiverem cobertas por testes, a API retornar erros previsíveis, o banco puder ser criado a partir das migrations e outra pessoa conseguir executar o projeto seguindo a documentação. A qualidade será avaliada pela clareza das decisões e pela capacidade do autor de explicar e alterar o código, não pela quantidade de bibliotecas.

1.5 Objetivo de portfólio

O projeto deve demonstrar profundidade no backend e competência prática suficiente no frontend para construir uma interface completa. O frontend não precisa ser uma experiência visual sofisticada; deve ser consistente, compreensível, responsivo e funcional.

2. Problema, usuários e limites

2.1 Problema

Negócios que trabalham com horário marcado precisam organizar a disponibilidade dos profissionais, evitar sobreposições, manter histórico dos atendimentos e registrar se os serviços foram pagos. Controles manuais ou planilhas podem permitir reservas conflitantes, perda de histórico e divergência entre o preço atual e o preço combinado.

2.2 Usuários previstos

- Operador/administrador do negócio: mantém cadastros e administra a agenda.
- Profissional: tem expediente definido e atende agendamentos associados a ele.
- Cliente: é a pessoa vinculada ao agendamento e ao histórico de atendimento.

Para manter o escopo pequeno, a primeira versão pode ser operada por uma pessoa responsável, usando Swagger e/ou a interface administrativa. Autenticação e autorização completas não são requisito obrigatório do MVP, a menos que sejam adicionadas explicitamente como uma fase separada.

2.3 Dentro do escopo

- CRUD e desativação de clientes, profissionais e serviços.
- Criação, consulta, reagendamento, cancelamento e conclusão de agendamentos.
- Cálculo automático do fim do atendimento e prevenção de conflitos por profissional.
- Registro e consulta do pagamento principal associado ao agendamento.
- Consultas simples de agenda, histórico e relatórios agregados.
- Testes, migrations, documentação Swagger e execução reproduzível.
- Frontend React com formulários, listagens, filtros e feedback de operação.

2.4 Fora do escopo inicial

- Pagamento online real, integração com gateway, estornos bancários ou armazenamento de dados de cartão.
- Envio de SMS, WhatsApp, e-mail, lembretes ou notificações push.
- Multi-tenancy, múltiplas unidades/filiais, recorrência e reservas em grupo.
- Aplicativo móvel nativo, microserviços, mensageria distribuída e Kubernetes.
- Autenticação complexa, recuperação de senha e gestão avançada de permissões; podem ser avaliadas após o núcleo.
- Grade semanal individual de expediente por dia da semana: a primeira versão usa uma janela diária por profissional, conforme o modelo de cinco tabelas.

3. Requisitos

3.1 Requisitos funcionais (RF)

ID	Requisito
RF-01	Cadastrar cliente com nome, e-mail e telefone obrigatórios.
RF-02	Consultar, editar e desativar cliente sem apagar seu histórico.
RF-03	Impedir e-mails duplicados de clientes, ignorando diferenças de maiúsculas/minúsculas quando suportado pela política escolhida.
RF-04	Cadastrar, consultar, editar e desativar profissionais.
RF-05	Registrar expediente diário de início e fim de cada profissional.
RF-06	Cadastrar, consultar, editar e desativar serviços.
RF-07	Permitir informar nome, descrição opcional, duração em minutos e preço do serviço.
RF-08	Criar agendamento vinculando um cliente, profissional e serviço existentes e ativos.
RF-09	Calcular automaticamente a data/hora final com base na duração atual do serviço.
RF-10	Validar que o agendamento ocorre dentro do expediente do profissional.
RF-11	Impedir sobreposição de agendamentos ativos para o mesmo profissional.
RF-12	Consultar agendamentos por data, período, profissional, cliente e status.
RF-13	Reagendar agendamento elegível, recalculando o fim e validando expediente e conflitos.
RF-14	Cancelar agendamento elegível preservando o registro histórico.
RF-15	Concluir atendimento conforme as regras de estado e pagamento definidas.
RF-16	Registrar no máximo um pagamento principal por agendamento.
RF-17	Atualizar o estado do pagamento e registrar método, valor e data quando pago.
RF-18	Consultar histórico de agendamentos de um cliente.
RF-19	Exibir agenda diária ordenada por horário.
RF-20	Gerar relatório de faturamento por período, serviços mais realizados e profissionais com mais atendimentos.
RF-21	Expor endpoints documentados via Swagger/OpenAPI.
RF-22	Disponibilizar uma interface React para os principais fluxos de cadastro, agenda e pagamento.
RF-23	Exibir estados de carregamento, sucesso, validação e erro no frontend.

3.2 Requisitos não funcionais (RNF)

ID	Área	Requisito
RNF-01	Manutenibilidade	Separar API, Application, Domain, Infrastructure e Tests por responsabilidade.
RNF-02	Integridade	Usar PKs, FKs, UNIQUE, NOT NULL e transações onde necessário.
RNF-03	Testabilidade	Regras críticas devem ser testáveis sem servidor web ou banco real quando forem testes unitários.
RNF-04	Consistência	Erros devem ter formato estável e códigos HTTP adequados.
RNF-05	Segurança	Não expor stack trace, segredos, strings de conexão ou dados sensíveis em respostas.
RNF-06	Reprodutibilidade	Banco e schema devem ser criados por migrations versionadas.
RNF-07	Usabilidade	Telas com rótulos claros, validação de formulário e feedback visível.
RNF-08	Responsividade	Interface utilizável em desktop e telas estreitas.
RNF-09	Documentação	README deve explicar dependências, configuração, migrations, execução e testes.
RNF-10	Qualidade	Build limpo, testes executáveis por comando e ausência de erros conhecidos nos fluxos principais.
RNF-11	Observabilidade	Registrar erros inesperados no servidor sem incluir segredos ou dados pessoais desnecessários.
RNF-12	Escopo	Não adicionar tecnologia sem uma necessidade técnica clara.

4. Regras de negócio (RN)

ID	Regra
RN-01	Cliente, profissional ou serviço inativo não pode ser usado em um novo agendamento.
RN-02	O e-mail do cliente deve ser único. A verificação no Service melhora a mensagem; a restrição UNIQUE no banco é a proteção final contra concorrência.
RN-03	O nome do serviço deve ser único conforme a política de comparação definida.
RN-04	A duração do serviço deve ser maior que zero.
RN-05	O preço do serviço não pode ser negativo.
RN-06	O expediente deve ter hora de início anterior à hora de fim.
RN-07	A data/hora final do agendamento é calculada no backend: início + duração do serviço. O cliente não define esse valor.
RN-08	O agendamento deve começar e terminar dentro do expediente diário do profissional.
RN-09	Para o mesmo profissional, dois agendamentos que não estejam cancelados não podem se sobrepor. Intervalos adjacentes são permitidos: 10:00–10:30 e 10:30–11:00.
RN-10	A detecção de sobreposição deve considerar intervalos, não apenas igualdade entre horários de início.
RN-11	A verificação de conflito precisa ser segura sob requisições simultâneas. Uma consulta seguida de INSERT sem estratégia de concorrência pode permitir corrida.
RN-12	O preço aplicado é uma cópia do preço do serviço no momento da reserva; alterações posteriores no serviço não mudam reservas antigas.
RN-13	Estados permitidos para agendamento: AGENDADO, CONFIRMADO, CONCLUIDO, CANCELADO e NAO_COMPARECEU.
RN-14	Agendamento CANCELADO ou CONCLUIDO é imutável quanto a reagendamento nesta versão.
RN-15	Transições de status inválidas devem ser recusadas pelo backend.
RN-16	Cada agendamento pode ter zero ou um pagamento principal. A FK idAgendamento deve ser UNIQUE em Pagamento.
RN-17	O valor de pagamento não pode ser negativo; para estado PAGO, valor deve ser positivo e dataPagamento deve estar preenchida.
RN-18	Um agendamento só pode ser considerado financeiramente concluído quando possuir pagamento PAGO, conforme a regra de conclusão escolhida.
RN-19	Operações que alterem agendamento e pagamento juntas devem usar transação quando a consistência exigir atomicidade.
RN-20	Registros com histórico devem ser desativados ou ter seu estado alterado, em vez de serem apagados fisicamente.
RN-21	O backend é a autoridade final de validação. Validações no React servem para orientar a pessoa usuária, não para garantir integridade.
RN-22	O fuso horário deve ser definido explicitamente. Para o MVP, documentar se a aplicação opera em horário local do negócio e padronizar conversões na API e no banco.

5. Arquitetura e organização

5.1 Modelo arquitetural

Monólito modular em uma única solution e uma aplicação implantável. A divisão em projetos organiza responsabilidades, mas não transforma o sistema em microserviços. O frontend React é uma aplicação cliente separada durante desenvolvimento e pode ser hospedado separadamente na entrega.

Projeto	Responsabilidade
AgendaPro.API	Controllers/endpoints, configuração HTTP, injeção de dependência, Swagger, middleware global de erros e composição da aplicação.
AgendaPro.Application	Casos de uso/Services, DTOs, validações, interfaces de persistência e coordenação das operações.
AgendaPro.Domain	Entidades, enums, invariantes e conceitos centrais sem dependência de ASP.NET ou EF Core.
AgendaPro.Infrastructure	DbContext, configurações EF Core, implementações de repositório, migrations e acesso ao MySQL.
AgendaPro.Tests	Testes unitários e testes de integração organizados por comportamento e camada.

5.2 Direção das dependências

- Domain não depende dos demais projetos.

- Application depende de Domain e define abstrações necessárias aos casos de uso.
- Infrastructure implementa as abstrações de persistência definidas pela Application e referencia Domain.
- API referencia Application e Infrastructure para registrar implementações na injeção de dependência.
- Tests referencia os projetos que precisa testar; testes de integração podem inicializar a API e um banco de teste.

5.3 Organização sugerida

```
AgendaPro.sln
src/AgendaPro.API/
src/AgendaPro.Application/
src/AgendaPro.Domain/
src/AgendaPro.Infrastructure/
tests/AgendaPro.Tests/
frontend/agenda-pro-web/
docker-compose.yml
README.md
```

5.4 Decisões de arquitetura

- Não criar Repository genérico complexo sem necessidade. Interfaces e implementações devem resolver necessidades reais.
- Controllers devem traduzir HTTP e delegar regras aos casos de uso; não concentrar regra de negócio.
- Evitar retornar entidades de persistência diretamente quando isso acoplar o contrato HTTP ao banco.
- Usar DTOs explícitos para criação, atualização e resposta.
- Usar CancellationToken em operações assíncronas relevantes quando isso estiver alinhado à implementação.
- Centralizar tratamento de exceções e evitar try/catch repetitivo em todos os controllers.

6. Modelo de dados — exatamente cinco tabelas

O modelo abaixo preserva o desenho inicial. Os nomes das colunas serão padronizados em camelCase no documento; o código C# pode usar PascalCase e o mapeamento EF Core fará a correspondência. Tipos e convenções finais devem ser confirmados ao implementar as migrations.

6.1 Cliente

Campo	Tipo sugerido	Restrições/observações
idCliente	INT	PK, AUTO_INCREMENT
nome	VARCHAR(120)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
telefone	VARCHAR(20)	NOT NULL
ativo	BOOLEAN	NOT NULL, DEFAULT TRUE
dataCadastro	DATETIME	NOT NULL, definido pelo backend

6.2 Profissional

Campo	Tipo sugerido	Restrições/observações
idProfissional	INT	PK, AUTO_INCREMENT
nome	VARCHAR(120)	NOT NULL
email	VARCHAR(150)	NULL, UNIQUE se informado
telefone	VARCHAR(20)	NULL
ativo	BOOLEAN	NOT NULL, DEFAULT TRUE
horaInicio	TIME	NOT NULL
horaFim	TIME	NOT NULL; maior que horaInicio

6.3 Servico

Campo	Tipo sugerido	Restrições/observações
idServico	INT	PK, AUTO_INCREMENT
nome	VARCHAR(100)	NOT NULL, UNIQUE conforme regra
descricao	VARCHAR(255)	NULL
duracaoMinutos	INT	NOT NULL, > 0
preco	DECIMAL(10,2)	NOT NULL, >= 0
ativo	BOOLEAN	NOT NULL, DEFAULT TRUE

6.4 Agendamento

Campo	Tipo sugerido	Restrições/observações
idAgendamento	INT	PK, AUTO_INCREMENT
idCliente	INT	FK → Cliente, NOT NULL
idProfissional	INT	FK → Profissional, NOT NULL
idServico	INT	FK → Servico, NOT NULL
dataHoraInicio	DATETIME	NOT NULL
dataHoraFim	DATETIME	NOT NULL, calculado no backend
precoAplicado	DECIMAL(10,2)	NOT NULL, snapshot do preço
status	VARCHAR(20) / enum	NOT NULL; valores documentados
observacao	VARCHAR(255)	NULL
dataCriacao	DATETIME	NOT NULL, definido pelo backend

6.5 Pagamento

Campo	Tipo sugerido	Restrições/observações
idPagamento	INT	PK, AUTO_INCREMENT
idAgendamento	INT	FK → Agendamento, NOT NULL, UNIQUE
valor	DECIMAL(10,2)	NOT NULL, >= 0
dataPagamento	DATETIME	NULL até o pagamento
metodo	VARCHAR(20) / enum	PIX, DINHEIRO, CARTAO, OUTRO
status	VARCHAR(20) / enum	PENDENTE, PAGO, CANCELADO

6.6 Relacionamentos

- Cliente 1:N Agendamento.

- Profissional 1:N Agendamento.
- Servico 1:N Agendamento.
- Agendamento 1:0..1 Pagamento; a restrição UNIQUE em Pagamento.idAgendamento impede pagamentos principais duplicados.

6.7 Índices e integridade

- Índices para Agendamento(idProfissional, dataHoraInicio), Agendamento(idCliente, dataHoraInicio), Agendamento(status, dataHoraInicio) e FKs.
- UNIQUE para Cliente.email, Profissional.email quando não nulo e Servico.nome conforme política definida.
- A regra de sobreposição não é resolvida por um UNIQUE simples em dataHoraInicio; exige validação por intervalo e estratégia contra concorrência.
- Usar DECIMAL para dinheiro, nunca float/double como tipo de persistência monetária.
- Definir política de exclusão de FKs: preferir Restrict/NoAction quando há histórico; desativar registros em vez de cascatas destrutivas.

7. Contrato da API

7.1 Convenções

- Prefixo de rotas sugerido: /api.
- JSON em requests e responses.
- DTOs de entrada diferentes de entidades e DTOs de resposta.
- Paginação pode ser simples ou adiada; se a listagem for sem paginação no MVP, documentar essa limitação.
- Swagger/OpenAPI deve mostrar operações, parâmetros, schemas e respostas esperadas.

7.2 Endpoints sugeridos

Método	Rota	Finalidade
GET	/api/clientes	Listar clientes; filtros opcionais e ativos.
GET	/api/clientes/{id}	Buscar cliente; 404 se inexistente.
POST	/api/clientes	Cadastrar cliente; 201, 400 ou 409.
PUT	/api/clientes/{id}	Atualizar cliente; 200, 400, 404 ou 409.
PATCH	/api/clientes/{id}/desativar	Desativar cliente sem apagar histórico.
GET/POST	/api/profissionais	Listar e cadastrar profissionais.
GET/PUT	/api/profissionais/{id}	Consultar e atualizar profissional.
PATCH	/api/profissionais/{id}/desativar	Desativar profissional.
GET/POST	/api/servicos	Listar e cadastrar serviços.
GET/PUT	/api/servicos/{id}	Consultar e atualizar serviço.
PATCH	/api/servicos/{id}/desativar	Desativar serviço.
GET/POST	/api/agendamentos	Consultar e criar agendamentos.
GET/PUT	/api/agendamentos/{id}	Consultar e reagendar quando permitido.
POST	/api/agendamentos/{id}/cancelar	Cancelar mantendo histórico.
POST	/api/agendamentos/{id}/concluir	Concluir conforme regras de pagamento.
GET	/api/agendamentos/agenda?data=YYYY-MM-DD	Agenda diária ordenada por início.
GET	/api/clientes/{id}/historico	Histórico do cliente.
GET/POST	/api/agendamentos/{id}/pagamento	Consultar/criar pagamento principal.
PUT	/api/pagamentos/{id}	Atualizar status/valor/método, respeitando transições.
GET	/api/relatorios/faturamento?inicio=&fim;=	Faturamento para pagamentos válidos no período.
GET	/api/relatorios/servicos-mais-realizados	Contagem de atendimentos por serviço.
GET	/api/relatorios/profissionais	Atendimentos por profissional.

7.3 Códigos HTTP

Código	Quando usar
200 OK	Consulta ou atualização concluída com resposta.
201 Created	Recurso criado; preferir Location quando houver URI de consulta.
204 No Content	Operação concluída sem corpo, como desativação/cancelamento quando adequado.
400 Bad Request	JSON inválido, campos ausentes ou formato inválido.
404 Not Found	Recurso solicitado não existe.
409 Conflict	E-mail/nome duplicado, conflito de horário ou transição de estado incompatível.
422 Unprocessable Entity	Opcionalmente, dados bem formados que violam regra de negócio; usar de forma consistente.
500 Internal Server Error	Falha inesperada; sem stack trace nem detalhes internos no response.

7.4 Formato de erro

Definir um ErrorResponse estável contendo timestamp, status HTTP, código interno, mensagem segura e, quando aplicável, lista de erros de validação. A mensagem para o cliente deve ser útil sem revelar SQL, stack trace, connection strings ou dados internos.

7.5 Swagger e testes manuais

Swagger é a ferramenta de exploração manual do contrato HTTP: permite selecionar endpoints, preencher payloads e inspecionar status e respostas. Não substitui testes automatizados. Manter exemplos de payloads válidos e inválidos para os endpoints mais importantes.

8. Estratégia de testes

8.1 Camadas de teste

Tipo	Objetivo	Exemplos
Unitário	Testar regras isoladas sem depender de servidor ou MySQL real.	Cálculo de fim, preço snapshot, validações, transições de status.
Integração	Testar componentes trabalhando juntos, especialmente API + persistência.	POST cria recurso, FK inválida, migrations e respostas HTTP.
Manual via Swagger	Explorar endpoints durante o desenvolvimento e depurar contratos.	Enviar requests, revisar status, headers e JSON.
Frontend	Validar formulários e estados visuais manualmente; automatizar testes de UI se couber no escopo.	Erro de validação, carregamento, lista vazia, sucesso e falha de API.

8.2 Casos de teste unitário — agendamento

ID	Cenário	Resultado esperado
UT-AG-01	Criar com cliente/profissional/serviço ativos e sem conflito	Agendamento criado; fim e preço calculados.
UT-AG-02	Serviço com duração de 45 min e início 10:00	Fim calculado como 10:45.
UT-AG-03	Preço do serviço alterado depois da reserva	precoAplicado do agendamento antigo permanece igual.
UT-AG-04	Cliente inativo	Criação recusada.
UT-AG-05	Profissional inativo	Criação recusada.
UT-AG-06	Serviço inativo	Criação recusada.
UT-AG-07	Fim depois do fim do expediente	Criação recusada.
UT-AG-08	Novo intervalo sobrepõe o início de outro	Conflito recusado.
UT-AG-09	Novo intervalo fica contido em outro	Conflito recusado.
UT-AG-10	Novo intervalo contém outro	Conflito recusado.
UT-AG-11	Novo intervalo começa exatamente quando outro termina	Permitido, se intervalos forem tratados como [início, fim).
UT-AG-12	Agendamento existente está CANCELADO	Não bloqueia o horário, conforme política escolhida.
UT-AG-13	Reagendar agendamento CONCLUIDO	Recusado.
UT-AG-14	Cancelar agendamento já cancelado	Comportamento definido e testado; preferir resposta idempotente ou conflito documentado.
UT-AG-15	Duas solicitações concorrentes para o mesmo profissional e intervalo	No máximo uma deve ser confirmada; estratégia de concorrência verificada por teste de integração.

8.3 Casos de teste unitário — clientes, profissionais e serviços

ID	Cenário	Resultado esperado
UT-CLI-01	Cadastrar cliente válido	Sucesso e dataCadastro definida no backend.
UT-CLI-02	E-mail vazio ou malformado	Validação recusada.
UT-CLI-03	E-mail duplicado	Conflito tratado de forma previsível.
UT-PRO-01	horaInicio anterior a horaFim	Profissional válido.
UT-PRO-02	horaInicio igual ou posterior a horaFim	Validação recusada.
UT-SRV-01	Duração zero/negativa	Validação recusada.
UT-SRV-02	Preço negativo	Validação recusada.
UT-SRV-03	Nome duplicado	Conflito tratado de forma previsível.
UT-DEL-01	Desativar entidade com histórico	Histórico preservado; entidade não aceita novos agendamentos.

8.4 Casos de teste — pagamentos e estados

ID	Cenário	Resultado esperado
UT-PAG-01	Criar pagamento para agendamento inexistente	Recusado como 404/erro de domínio mapeado.
UT-PAG-02	Criar segundo pagamento principal para o mesmo agendamento	Conflito; UNIQUE também protege no banco.
UT-PAG-03	Marcar como PAGO com valor zero	Recusado.
UT-PAG-04	Marcar como PAGO sem dataPagamento	Recusado ou data definida pelo backend, conforme decisão documentada.
UT-PAG-05	Método fora dos valores permitidos	Recusado.
UT-PAG-06	Concluir atendimento sem pagamento PAGO	Recusado conforme RN-18.
UT-PAG-07	Erro ao salvar pagamento durante operação transacional	Nenhuma alteração parcial deve persistir.

8.5 Casos de teste de integração/API

ID	Cenário	Resultado esperado
IT-API-01	POST /api/clientes com dados válidos	201 Created e DTO de resposta.
IT-API-02	POST /api/clientes com e-mail duplicado	409 Conflict com formato padronizado.
IT-API-03	GET de ID inexistente	404 Not Found.
IT-API-04	POST agendamento com FK inexistente	Resposta controlada; nunca 500 inesperado.
IT-API-05	POST agendamento conflitante	409 Conflict.
IT-API-06	GET agenda diária	Ordenação por dataHoraInicio e filtros corretos.
IT-API-07	Erro inesperado simulado	500 seguro, log interno e sem stack trace na resposta.
IT-API-08	Aplicar migrations em banco vazio	Schema e restrições criados corretamente.
IT-API-09	Duas requisições simultâneas conflitantes	Apenas uma reserva confirmada.
IT-API-10	Operação que altera agendamento e pagamento falha no meio	Rollback transacional comprovado.

8.6 Princípios para testes

- Cada teste deve ter nome que descreva o cenário e o resultado esperado.
- Testar caminhos felizes e caminhos de falha; não testar apenas getters ou detalhes de implementação.
- Mockar dependências externas nos unitários quando isso tornar o teste focado; não usar mock para fingir que a integração com o banco está correta.
- Usar banco de teste isolado para testes de integração e nunca apontar testes destrutivos para o banco de desenvolvimento real.

- A estratégia contra concorrência deve ser validada em integração com o banco e o mecanismo escolhido; um teste unitário isolado não prova a atomicidade do banco.
- Executar a suíte completa antes de cada marco importante e registrar comandos no README.

9. Frontend React com JavaScript

9.1 Objetivo

Construir a interface depois que os contratos principais da API estiverem estáveis. O foco é aprender React fazendo: componentes, props, estado, eventos, formulários controlados, efeitos, chamadas HTTP, roteamento, CSS e tratamento de erros.

9.2 Telas mínimas

Tela	Responsabilidade
Visão geral	Resumo da agenda do dia e atalhos para operações frequentes.
Clientes	Listar, pesquisar, cadastrar, editar e desativar clientes.
Profissionais	Listar, cadastrar, editar, desativar e mostrar expediente.
Serviços	Listar, cadastrar, editar e desativar serviços.
Agenda	Visualizar agendamentos por dia e filtrar por profissional/status.
Novo agendamento	Selecionar cliente, profissional, serviço e horário; mostrar fim e preço calculados pela API.
Detalhes do agendamento	Ver dados, reagendar, cancelar e concluir conforme estado.
Pagamento	Registrar valor, método e status do pagamento.
Relatórios	Exibir faturamento por período e serviços/profissionais mais realizados.

9.3 Regras de implementação do frontend

- Usar React + JavaScript, sem TypeScript no escopo inicial.
- Manter componentes pequenos e com responsabilidade clara; evitar um único componente gigante.
- Separar funções de comunicação HTTP dos componentes de tela.
- Não duplicar regra de negócio crítica: o frontend pode antecipar validações, mas a API confirma tudo.
- Exibir carregamento, sucesso, lista vazia, erros de validação e falhas de rede.
- Usar labels associadas aos inputs, foco visível, botões com nomes claros e layouts que funcionem em telas menores.
- Evitar dependências de UI pesadas; priorizar CSS próprio e componentes reutilizáveis.
- Não guardar segredos no frontend. Se autenticação for adicionada, definir fluxo e armazenamento conscientemente.

9.4 Estratégia de aprendizagem

1. Estudar fundamentos de HTML semântico, CSS layout (Flexbox/Grid) e responsividade.
2. Praticar componentes, props, estado e eventos em uma tela pequena.
3. Implementar um CRUD de ponta a ponta antes de replicar o padrão.
4. Integrar a API com tratamento de loading/error/success.
5. Construir a agenda e o formulário de agendamento, que são as telas de maior valor de domínio.
6. Finalizar responsividade, acessibilidade básica e polimento visual.

10. Tecnologias e ferramentas

Área	Tecnologia	Uso
Backend	C#, .NET / ASP.NET Core Web API	Endpoints HTTP, DI, configuração, middleware e documentação.
Arquitetura	Monólito modular em projetos .NET	Separação de responsabilidades sem microserviços.
Persistência	Entity Framework Core	Mapeamento, consultas e migrations.
Banco	MySQL	Banco relacional principal.
API docs/manual	Swagger / OpenAPI	Exploração e teste manual de endpoints.
Testes	xUnit (sugestão)	Testes unitários; ferramentas adicionais podem ser escolhidas conforme necessidade.
Mocking	Moq ou NSubstitute (opcional)	Simular dependências em testes unitários quando fizer sentido.
Integração	WebApplicationFactory e banco de teste	Validar pipeline HTTP e persistência realista.
Frontend	React + JavaScript	Interface web.
Build frontend	Vite	Servidor local e build do frontend.
HTTP frontend	fetch nativo ou Axios	Comunicação com a API; escolher uma opção e manter consistência.
Versionamento	Git e GitHub	Histórico, branches quando úteis, issues e releases.
Ambiente	Docker e Docker Compose	Reproduzir execução da API e do MySQL.

10.1 Migrations

- Configurar entidades, chaves, índices e relacionamentos no EF Core.
- Criar a primeira migration a partir do modelo aprovado.
- Aplicar migration em banco de desenvolvimento e conferir schema.
- Versionar arquivos de migration no Git.
- Criar novas migrations para mudanças de schema; não corrigir o banco manualmente sem registrar a mudança no modelo.
- Testar a criação do schema em um banco vazio antes de considerar o projeto reproduzível.

10.2 Docker

Docker é uma fase posterior, depois de a API e o banco funcionarem localmente. O Compose deve iniciar os serviços necessários com configuração por variáveis de ambiente. Credenciais de desenvolvimento devem ser documentadas como exemplos e não confundidas com segredos de produção.

11. Plano de desenvolvimento

Fase	Entregável
Fase 0 — Planejamento e Git	Criar repositório, README inicial, .gitignore, definir nomes, escopo e convenções.
Fase 1 — Solution e dependências	Criar os cinco projetos, configurar referências e confirmar build limpo.
Fase 2 — Domain	Criar cinco entidades, enums/constantes, nullability e invariantes.
Fase 3 — MySQL e migrations	Configurar EF Core, DbContext, mapeamentos, FKs, índices, UNIQUE e primeira migration.
Fase 4 — Persistência	Implementar consultas e gravações necessárias; manter abstrações simples.
Fase 5 — DTOs e validação	Criar requests/responses, validações de entrada e contratos claros.
Fase 6 — Services básicos	Implementar CRUDs e desativação de clientes, profissionais e serviços.
Fase 7 — Núcleo de agendamento	Implementar cálculo do fim, expediente, intervalos, conflito e reagendamento.
Fase 8 — Estados e pagamento	Implementar cancelamento, conclusão, pagamento, consistência e transações.
Fase 9 — Controllers e Swagger	Publicar endpoints, códigos HTTP e exemplos de payload.
Fase 10 — Testes	Cobrir regras unitárias e cenários de integração prioritários.
Fase 11 — Relatórios SQL	Agenda diária, histórico, faturamento e agregações.
Fase 12 — Erros e logging	Middleware global, ErrorResponse, logs e mapeamento de exceções.
Fase 13 — Frontend	Estudar React/JS e implementar telas em etapas, começando por um CRUD.
Fase 14 — Docker e documentação	Executar em ambiente limpo, atualizar README e documentar comandos.
Fase 15 — Revisão final	Rodar build/testes, revisar segurança, nomes, dependências e criar release.

11.1 Definição de pronto por funcionalidade

- Contrato e validações definidos.
- Regra implementada no lugar correto.
- Teste unitário para comportamento relevante.
- Teste de integração quando houver interação importante com banco/API.
- Endpoint documentado no Swagger.
- Resposta e erros revisados manualmente.
- Frontend integrado, quando a funcionalidade tiver tela.
- Commit feito com mensagem clara.

11.2 Sugestão de estratégia de commits

- Um commit por incremento coerente, por exemplo: feat: add client persistence.
- Separar refatoração de alteração funcional quando possível.
- Não commitar binários, credenciais, arquivos .env reais ou dados pessoais.
- Executar build e testes antes de commits que marquem um marco funcional.

12. Git, README e entrega

12.1 Checklist inicial do repositório

- Criar repositório público ou privado conforme a estratégia de portfólio.
- Adicionar .gitignore para .NET, Node, IDE e arquivos de ambiente.
- Criar .env.example ou exemplos de configuração sem senhas reais.
- Adicionar README com problema, funcionalidades, arquitetura, stack, pré-requisitos e comandos.
- Adicionar licença se fizer sentido para o objetivo de publicação.

- Usar issues/checklists para dividir fases grandes em tarefas pequenas.

12.2 Conteúdo obrigatório do README

- Resumo do projeto e problema que resolve.
- Funcionalidades principais e regras relevantes.
- Diagrama simples da arquitetura e descrição dos projetos.
- Modelo de dados e relacionamentos.
- Tecnologias e versões necessárias.
- Como configurar conexão com MySQL e aplicar migrations.
- Como executar a API, abrir Swagger, executar testes e iniciar frontend.
- Como iniciar com Docker, quando disponível.
- Variáveis de ambiente necessárias, sem valores secretos.
- Decisões técnicas, limitações conhecidas e próximos passos.
- Capturas de tela do frontend após sua conclusão.

12.3 Critérios finais de aceite

Área	Critério
Produto	CRUDs essenciais e fluxos de agendamento/pagamento funcionam.
Dados	Banco criado por migrations; PKs, FKs e restrições verificadas.
Regras	Conflito de horário e transições inválidas são recusados.
Testes	Testes unitários passam; testes de integração cobrem fluxos críticos.
API	Swagger atualizado; status HTTP e erros coerentes.
Frontend	Telas funcionais, responsivas e com estados de erro/sucesso.
Execução	Projeto pode ser iniciado seguindo README; Docker quando finalizado.
Portfólio	Git organizado, documentação clara e autor consegue explicar as decisões.

13. Riscos, decisões em aberto e recomendações

13.1 Riscos principais

- Conflito simultâneo: consultar disponibilidade e depois salvar não basta; escolher e testar uma estratégia de concorrência com o MySQL.
- Fuso horário: DATETIME não carrega fuso. Documentar o fuso adotado e padronizar o comportamento do frontend e backend.
- Estados ambíguos: documentar quais transições são válidas para agendamento e pagamento antes de criar endpoints.
- Preço e duração: guardar snapshot do preço e decidir se a duração também precisa ser copiada para preservar a duração original caso o serviço seja alterado depois. Recomendação: considerar armazenar duracaoAplicada no agendamento, se a regra exigir histórico fiel; isso adiciona uma coluna, não uma tabela.
- Relatório de faturamento: definir se o período usa dataPagamento ou data do atendimento; recomendação inicial: usar dataPagamento para faturamento recebido.
- Testes de banco: testes unitários com mocks não comprovam constraints, migrations ou concorrência; complementar com integração.
- Autenticação: decidir explicitamente se fica fora do MVP. Se incluída, planejar identidade, senha com hash, autorização e testes como um escopo separado.

13.2 Decisões a fechar antes de implementar

ID	Decisão	Orientação inicial
D-01	Fuso horário do negócio	Definir uma zona horária única e como converter datas na API e na interface.
D-02	Política de conflito	Definir se CANCELADO e NAO_COMPARECEU liberam o horário; recomendação: cancelado libera, concluído e não compareceu preservam histórico sem conflito futuro.
D-03	Conclusão e pagamento	Confirmar se conclusão exige pagamento PAGO ou se pode haver atendimento concluído e pagamento pendente. O documento adota a primeira regra inicialmente.
D-04	Duração aplicada	Decidir se Agendamento terá coluna duracaoAplicada para preservar também a duração original.
D-05	Autenticação	Manter fora do MVP ou adicionar depois como fase opcional.
D-06	Relatórios	Definir o que conta como atendimento realizado e qual data determina o faturamento.
D-07	Concorrência	Escolher transação/isolamento e/ou mecanismo de serialização/lock apropriado e provar com teste de integração.

13.3 Regra de ouro

Não adicionar tecnologia para aumentar a lista do currículo. Cada abstração, biblioteca e camada precisa resolver um problema que o autor consiga explicar. O projeto é concluído quando funciona, pode ser testado e documentado, e o autor entende suas decisões.

Apêndice A — Payloads ilustrativos

Criar cliente

```
{
"nome": "Ana Souza",
"email": "ana@example.com",
"telefone": "11999990000"
}
```

Criar agendamento

```
{
"idCliente": 1,
"idProfissional": 2,
"idServico": 3,
"dataHoraInicio": "2026-11-02T10:00:00",
"observacao": "Preferência por atendimento tranquilo"
}
```

O payload de agendamento não inclui dataHoraFim, precoAplicado ou dataCriacao: esses valores são calculados/definidos pelo backend. O formato final dos DTOs deve ser documentado no Swagger.

Apêndice B — Checklist de início

- Repositório criado e .gitignore revisado.
- README inicial escrito.
- Solution criada e todos os projetos compilam.
- Referências entre projetos revisadas.
- Entidades e regras iniciais documentadas.
- MySQL local configurado sem credenciais no Git.
- Primeira migration criada e aplicada em banco vazio.
- Primeiro teste unitário executado com sucesso.
- Primeiro endpoint aparece no Swagger.